BROWSER vs MACHINE vs AGENT — THREE CALLERS, THREE WAYS TO FAIL

who is calling, what proves it, what limits it, and where each one breaks

Dashed = a gap. B = browser, M = machine, A = agent. Numbers match the list below.

THREE CALLERS, ONE ENDPOINT — THE SAME API CAN'T TRUST THEM THE SAME WAY

Browser

a person, through a user agent you don't control

WHO CALLS
A person, plus every script on the page

PROVES IDENTITY WITH
Session cookie, OAuth or passkey login

CONSENT ARRIVES AS
A banner choice: Consent Mode v2 signals

LIMITS ENFORCED BY
Same-origin policy, CSP, SameSite cookies, CORS

GAPS

- **B1** Client checks are suggestions: anyone edits the JS
- **B2** Third-party tags see everything on the page
- **B3** Tokens in localStorage are readable by any XSS
- **B4** Tags fire before the consent choice lands
- **B5** Hydrated props ship data into the HTML

Machine

a service calling a service: webhooks, cron, ETL, iPaaS

WHO CALLS
A service account, often with no person behind it

PROVES IDENTITY WITH
API key, HMAC signature, mTLS or client credentials

CONSENT ARRIVES AS
Nothing, unless the record carries it as data

LIMITS ENFORCED BY
Scopes, network policy, allowlists, rate limits

GAPS

- **M1** Long-lived keys with whole-store scopes
- **M2** No person in the loop: the job runs with full power
- **M3** Consent drops out of the record in transforms
- **M4** Retries and redelivery replay writes
- **M5** Queues and logs keep the payload

Agent

an AI acting for a person: MCP tools, agentic checkout

WHO CALLS
A model choosing tools, for a user who isn't watching

PROVES IDENTITY WITH
A delegated token plus a signed mandate (AP2)

CONSENT ARRIVES AS
A mandate per action: what, how much, until when

LIMITS ENFORCED BY
Tool allowlist, mandate check, budget, approval

GAPS

- **A1** Prompt injection: content becomes instructions
- **A2** Excessive agency: tools broader than the task
- **A3** Token passthrough: the agent forwards your token
- **A4** Mandate replay, or scope drift past the mandate
- **A5** Exfiltration through a tool call or a URL
- **A6** Same prompt, different action: retries double-spend
- **A7** No record of why the agent acted

Your endpoint: Cloudflare Worker · Xano · API

First ask which kind of caller this is, then apply that caller's rule. One rule for all three fails all three.

Browser: a user session, CSRF protection, and server-side authorization per record
Machine: a verified signature or mTLS, a narrow scope, and an idempotency key
Agent: the user AND the agent identified, a valid mandate, an allowed tool, and an audit entry

WHERE EACH ONE FAILS

CONCERN	BROWSER	MACHINE	AGENT
Fails open when	A check lives only in client JavaScript	A key outlives its job or its scope	Tool output is read as an instruction
Replay looks like	A resubmitted form or a stolen session	A redelivered webhook or a re-run batch	A retried tool call or a reused mandate
Data leaks through	Third-party tags, page source, caches	Queues, logs, iPaaS error records	Prompts, tool calls, model context
Consent is checked	Before any tag loads	On every record, as a field	Before every side effect, as a mandate
Audit must answer	Which user, which session?	Which key, which job, which payload?	Which user, agent, mandate and tool, and why?

ASSUMPTIONS THAT BREAK AS YOU GROW — FROM A JVM THAT NEVER LEAVES THE VM TO A SYSTEM AI CAN READ

ASSUMPTION	A JVM THAT NEVER LEAVES THE VM	WHEN YOU ADD COMMERCE	WHEN YOU GO GLOBAL	WHEN YOU ADD LLMS: AI-LEGIBLE
Trust boundary	The VM is the wall; inside is trusted	Payments, tax and tag scripts run outside it	Clouds and regions multiply the walls	Your content is input to someone else's model
Identity	One app, one user table, one session store	Customers, guests, staff, payment tokens	Local logins and age rules per market	An agent acts for a user: identify both
Where data lives	In the heap and one database	Card data enters PCI scope; webhooks leave	Residency: where each copy lives is legal	Prompts, embeddings and model logs are copies
Consent	Not modeled: the app owns its data	Marketing consent and order data diverge	GDPR, CPRA, PIPA, CASL: defaults differ	Training and retrieval need their own yes
Time and money	One clock, one currency, one locale	Retries and refunds need idempotency keys	Currencies, tax, FX, time zones, languages	An agent can spend: mandate, amount, expiry
Failure	An exception stays in the log	A failure can charge twice or lose an order	One region down mustn't take the others	A plausible wrong answer looks like success
Audit asks	Who logged in?	Who paid, refunded or changed a price?	Which law applied, and where did data go?	Which user, agent, mandate and source?
Readable by	Your own team	Auditors, the PCI assessor, payment partners	Regulators in every market	Crawlers, answer engines and agents

Each column keeps every problem to its left. The AI column is dashed: making a system legible to machines also makes it readable, quotable and steerable by them.

SEVENTEEN GAPS — AND THE CONTROL THAT CLOSES EACH

B1 Client checks are suggestions
Fix: Authorize every request on the server, per record

B2 Third-party tags see the page
Fix: CSP, Subresource Integrity, a script inventory (PCI DSS 6.4.3)

B3 Tokens readable by XSS
Fix: HttpOnly, Secure, SameSite cookies; nothing in localStorage

B4 Tags fire before consent
Fix: Consent Mode default denied; load tags after the choice

B5 Hydration ships data
Fix: Pass only what the screen shows; fetch PII after consent

M1 Keys too broad, too long
Fix: Least-privilege scopes, short lifetimes, rotation

M2 No person in the loop
Fix: Act as a named service, log it, cap what it can change

M3 Consent drops out
Fix: Carry consent state as a field; refuse records without it

M4 Replays duplicate writes
Fix: Verify the HMAC on the raw body; one idempotency key per event

M5 Payloads persist
Fix: Store IDs, not payloads; purge queues; scrub logs

A1 Prompt injection
Fix: Treat fetched content as data; approve side effects outside the model

A2 Excessive agency
Fix: One narrow tool per task; read and write as separate tools

A3 Token passthrough
Fix: Audience-bound tokens per tool; never forward the user's token

A4 Mandate replay or drift
Fix: Sign amount, merchant, expiry and nonce; check every call

A5 Exfiltration by tool
Fix: Egress allowlist; no raw-URL tools; filter outputs for PII

A6 Retries double-spend
Fix: An idempotency key on every side-effecting tool call

A7 No record of why
Fix: Log user, agent, mandate, tool and inputs to an append-only ledger

A browser is a person whose code you can't trust. A machine is code with no person. An agent is code choosing what to do for a person. Each needs its own rule at the same endpoint.